OSS-Next Completion Audit — Task List

Generated 2026-07-21 · Branch `integrate-wp-render` · HEAD `3d0d96f`
Living source: `docs/audits/AUDIT_REQUIREMENTS.md` §6 · Previous export: `AUDIT_TASK_LIST_2026-07-18.pdf`

> **This was an event-driven pass, not a full three-dimension re-run.** It records what changed while integrating the converted-pages renderer and reacting to two backend changes. Treat the scores as a delta on 2026-07-18, not an independent re-scoring. A full methodology pass (§3 of the requirements doc) is still owed.

Scores

Dimension	2026-07-21	07-18	07-17	07-15
Overall (blended)	**~81%**	~79%	~75%	~65%
Functional Completeness (8 areas)	~88%	~87%	~81%	~62%
E-Commerce Feature Coverage (41 items)	~64%	~62%	~62%	~65%
Core Web Vitals / PageSpeed / SEO	~91%	~89%	~82%	~68%

What moved, and why:

- **Feature Coverage +2** — "reviews & ratings" goes 🟡 → ✅ on PLP and PDP. The backend now indexes `ratings: { rating, review_count }`, so review counts are real instead of a hardcoded `0`.
- **CWV/SEO +2** — PDP `Product` JSON-LD now emits `aggregateRating` (blocked until `review_count` existed), and WordPress content pages became server-rendered and indexable instead of iframed.
- **Functional +1** — checkout submit-order fix, Braintree payer details and the PDP crash fix, offset by two newly-found defects.

> **Two findings this pass are more serious than the score movement suggests:** a client-controlled charge amount, and a charged-but-no-order state that the backend's checkout crash makes reachable on the very first real transaction. Both are High priority below.

Progress: 42 items complete · 12 open (5 high, 1 medium, 6 lower/needs product decision)

Open — High Priority

FRONTEND NEW `/api/braintree_checkout` charges a client-supplied amount

The route reads `amount` straight from the request body and passes it to `transaction.sale()`; nothing server-side reconciles it against the cart. A crafted request charges $0.01 for a $3,000 container. Pre-existing — not introduced by the payer-details change.
Fix before any live card processing: recompute the total server-side via the existing `/api/orders/get-total` (which already derives it from cart + shipping address) and charge that, ignoring the client's number. Harmless in sandbox; a real vulnerability the moment production credentials are set.

FRONTEND NEW A successful charge with a failed order-create shows a raw error

`handlePlaceOrder()` charges first, then POSTs `/api/orders/checkout`. If the second call fails, the customer has been charged with no confirmation and a raw error message. Not hypothetical — the Django checkout crash below fires on *every* call, so this is the state the first real sandbox transaction will land in.
Needs a dedicated "payment taken, order needs attention" screen carrying the `transaction_id`, plus a support path. Independent of the backend fix: network failures can produce this in production regardless.

BACKEND Django checkout endpoint crashes on every call

```
ValueError: Currency formatting is not possible using the 'C' locale.
app/orders/views.py:89 — locale.currency(float(item.total), grouping=True)
```

The server's OS locale isn't configured. **The order saves before the crash** — real order records were confirmed created despite every attempt 500ing. Once Braintree credentials are live, a customer would be charged, have their order created, and still see a broken page.
Fix: call `locale.setlocale()` at startup, or drop `locale.currency()` and format manually. Also unresolved: `payment_method: "braintree"` does not by itself move order status to `"paid"` — something else server-side controls that.

BACKEND NEW The Django pages API serves malformed CSS

The CSS minifier strips `/* */` delimiters but leaves the comment text behind, and emits unbalanced braces:

```
once slick adds its class,show it smoothly .shipping-container-slider…{…}
}.slick-slide:not(.slick-active){opacity:0!important}
```

Strict parsers throw; browsers error-recover by **discarding every rule after the error point**. Worked around here with `postcss-safe-parser`, but the same CSS is served to real browsers on the live WordPress site, which are silently losing styling nobody has noticed. Fix in the Django CSS pipeline.

FRONTEND NEW `middleware.ts` is deprecated in Next 16 — and renaming it broke the A/B test

The build warns to use `proxy.ts` instead. A direct rename (`middleware.ts` → `proxy.ts`, `export function middleware` → `proxy`) typechecks, lints and builds clean, but silently empties `middleware-manifest.json` — the homepage A/B/C variant assignment stops running entirely, with no error. Reverted rather than shipped.
Needs the actual cause found before renaming. **A green build is not sufficient evidence here.**

Open — Medium Priority

BLOCKED · CLIENT **Decide the homepage's direction — `QuoteForm` fakes a successful submit**

The homepage is almost entirely static marketing content. Worse than originally described: the submit button (`onClick={() => setSubmitted(true)}`) shows "✓ Quote Request Sent — We'll be in touch!" while the collected name/email/phone/ZIP/container details go nowhere. Actively misleading, not merely inert.
Planned fix is a **Zoho Forms** integration, but the client hasn't provided the Zoho account/form details. Not actionable until those arrive.

Open — Lower Priority / Needs a Product Decision

PRODUCT **Live chat — build it or remove the marketing claim**

`TrustStrip.tsx` advertises "Phone, Email & Chat" support, but no chat implementation exists anywhere. A false claim to customers today. Client hasn't decided on chat itself; revisit the *copy* specifically if the decision stalls much longer.

PRODUCT **Newsletter signup UI**

The largest genuinely unbuilt feature. Backend utilities already exist (`lib/newsletter.ts`), so each piece is "build UI and call this": footer widget, account-dashboard toggle, unsubscribe page. Still pending a client decision.

BACKEND **Clarify who owns order confirmation emails**

No email-sending code (SendGrid/nodemailer/SMTP) exists anywhere in this Next.js app. If confirmation emails are sent today it's entirely a Django responsibility, with zero visibility from here. Worth confirming rather than assuming it's covered.

FRONTEND **PLP cards never pass `priority` to `next/image`**

Investigated 2026-07-18 and **deliberately not fixed.** The cards render through react-instantsearch's `<Hits hitComponent>`, whose render prop exposes no item index — there's no clean way to identify the first card without replacing `<Hits>` with a manual `useHits()` + `.map()`, risking pagination/insights/empty-state regressions for a low-severity CWV nit that isn't on the SSR preload path anyway.

PRODUCT BACKEND **Address book is not a true multi-address book**

Exactly one billing + one shipping address baked into the profile (`ADDRESS_TYPES` is a fixed 2-entry array) — no add/remove, no multiple saved addresses, no default selection. Fine as "edit your one address." True multi-address support needs a product decision *and* backend support the profile endpoint doesn't have today.

PRODUCT BACKEND **No shipment/carrier tracking number in Order History**

`my-account/orders` shows real order status, but no `tracking_number`/carrier field exists anywhere (confirmed via grep — zero matches). Distinct from "order status updates," which is already real. Likely needs the backend to start returning it.

Closed This Pass — 2026-07-21

- **PDP crashed on every shipping-container page.** The backend changed `ratings` from a number to `{ rating, review_count }`; `ProductInfoPanel.tsx` threw `TypeError: rating.toFixed is not a function`. Fixed via `lib/ratings.ts`'s `normaliseRating()`, tolerant of the object, a bare number and a numeric string. Five call sites — **two failed silently rather than crashing**: the PLP card's `typeof === 'number'` guard scored every product 0 forever, and the `best_rated` sort ordered on a field that is no longer a number.
- **PLP ratings/review-count sync** — long-standing item, resolved. The referenced memory file `backend-reminder-plp-ratings.md` no longer exists; the pointer was dangling.
- **PDP `aggregateRating` JSON-LD** — previously impossible for want of a `reviewCount`. Emitted only when count > 0, since a zero-count aggregate is invalid structured data and Google treats empty/inflated ratings as a manual-action risk.
- **WordPress theme CSS repainted the app's header/footer.** Scoped every selector to a `.wp-content` wrapper. Found a worse bug while verifying: `postcss.parse` threw on the API's CSS and the `catch` silently dropped the entire 400KB stylesheet, rendering pages nearly unstyled *while the build stayed green*.
- **Converted WordPress pages render inline instead of iframed** — from the Django pages API, inside the app's own chrome, with LCP preloads and image lazy-loading. The old iframe proxy is deleted.
- **Place Order moved below the payment form it depends on.** The button sat above the card form, delivery confirmation and terms checkbox — all of which it validates. Its own error copy had to point users past itself ("… confirm delivery details below").
- **Payer details now sent to Braintree** — customer, billing and shipping, so transactions are identifiable in the control panel instead of appearing as a bare amount. Also enables AVS (no Braintree fee; ~$0.01 Mastercard card-not-present assessment).
- **`NEXT_SOLANA_*` → `NEXT_OSS_*` env rename finished.** `docs/reference/*` deliberately keeps the old name — those files document the original WordPress app and cite its `src/pages/api/*.js` paths.

Blocked — Not Actionable From This App

Item	Blocked on
Real checkout end-to-end	Django `locale.currency()` crash + Braintree credentials
Order status → `"paid"`	Backend — unclear what controls it
Guest abandoned-cart notify	Backend — guests get no `cart_id`; validation demands full billing address
Reviews edit-prefill / populated carousel	Backend — reviews appear moderated; `list` returns `count: 0` for new ones
Populated Order History rendering	Needs a real order, which needs the checkout crash fixed first
Order confirmation emails	Backend — ownership unconfirmed
Live WordPress page styling	Backend — malformed CSS from the pages API

Generated from `docs/audits/AUDIT_REQUIREMENTS.md` §5–§6, cross-checked against `API_INTEGRATION_STATUS.md` and `API_TRIGGER_CHECKLIST.md`. The `.md` files are the living versions — prefer them over this export. Prior exports retained for comparison: `AUDIT_TASK_LIST_2026-07-18.pdf` (root), `docs/audits/pdf/AUDIT_TASK_LIST_2026-07-17.pdf`.